

Software Architecture Document (SAD)

Snake Game

SE Mini Project - Team 3

Name	SRN	Contribution
Jaanhavi Ashwin Kher	PES2UG24CS199	1. Introduction, 2. Document Overview, 3.1 Architecture Goals & Constraints, 4.4 Error Handling, Logging & Monitoring
Krishna Prajoth G	PES2UG24CS237	3.3 Component (UML) Diagram, 3.4 Component Descriptions, 3.7 Risks & Mitigations, 3.8 Traceability to Requirements, 3.9 Security Architecture
Krithika S	PES2UG24CS238	3.2 Stakeholders & Concerns, 3.5 Chosen Architecture Pattern & Rationale, 3.6 Technology Stack & Data Stores, 4.6 Open Issues & Next Steps
Joey Fedrick K	PES2UG24CS206	4.1 Design Overview, 4.2 UML Sequence Diagrams, 4.3 API / Interface Design, 4.5 UX Design

Based on the approved Snake Game SRS v1.0.

1. Introduction

1.1 Purpose

This Software Architecture Document (SAD) defines the architecture of the Snake Game system. It describes how the system is structured internally - the components, their responsibilities, and how they interact - to deliver the functionality and quality attributes required of the game.

This document serves as the shared architectural reference for the development team during implementation, ensuring a single, consistent system structure across all modules.

1.2 Scope

This SAD covers the internal software architecture of the Snake Game - a standalone, single-player, offline desktop application implemented in C/C++ using SFML, with a console-based interface as an alternative build target. It does not cover network architecture, backend services, or database design, as the system has none.

1.3 Definitions and Terminology

Term	Definition
Player	The user who controls the Snake using keyboard input.
Snake	The controllable game entity that moves across the Game Board and grows when Food is collected.
Snake Head	The leading segment of the Snake, used to detect collisions with boundaries, the Snake's own body, and Food.
Snake Body	The trailing segments of the Snake that grow as Food is collected.
Food	An item placed at a valid random position on the Game Board that can be collected by the Snake.
Game Board	The grid-based area in which the Snake moves and Food is placed.
Score	The numerical value representing the Player's progress.
Game State	The current operating state of the game: Menu, Playing, Paused, or Game Over.
Menu	The initial Game State, from which the Player may start a new game or exit.
Playing	The Game State in which the Snake moves and core game logic is active.
Paused	The Game State in which gameplay is temporarily suspended.
Game Over	The Game State reached when the Snake collides with a boundary or with its own body.
Restart	The process of returning the game to its initial state after Game Over.

2. Document Overview

This document is organized into two parts.

Architecture establishes the system's architectural foundation: the goals and constraints guiding the design, the stakeholders and their concerns, the system's component structure, the chosen architecture pattern and rationale, the technology stack, known risks, traceability to requirements, and the security architecture.

Design builds on that architecture to describe the game's actual design: how the architecture translates into gameplay, the dynamic behavior of the system through sequence diagrams, the internal interfaces between components, error handling and logging, the user interface design, and open issues for future consideration.

The following nine components are used consistently throughout this document: **Game Controller, Input Handler, Snake Manager, Food Manager, Collision Manager, Score Manager, Game Renderer, Game State Manager, and Difficulty Manager**. The system operates across four Game States: **MENU, PLAYING, PAUSED, and GAME OVER**.

3. Architecture

3.1 Architecture Goals & Constraints

Architecture Goals

- **Maintainability:** The system uses a modular layered architecture with clearly separated components, so that each can be modified independently.
- **Performance:** The architecture supports input response within 100 ms under normal conditions, keeping the path from input capture to rendering lightweight.
- **Reliability:** The architecture maintains a consistent Game State during gameplay without unexpected crashes, through clear ownership of state and well-defined transitions.
- **Usability:** Control handling and Game State display are kept simple enough that the Player can identify controls and status without additional instruction.
- **Portability:** The architecture avoids platform-specific dependencies beyond the SFML and console build targets, so the application runs on the supported desktop environment.
- **Security:** The architecture protects the integrity of the game's internal state and prevents unsupported inputs or external interfaces from causing invalid behavior.

Architecture Principles

- Separation of concerns: Each component is responsible for a distinct aspect of game functionality.
- Single responsibility: Components perform only the operations associated with their defined responsibility.
- Controlled state transitions: Changes between Game States are managed through the Game State Manager.
- Loose coupling: Components communicate through defined interfaces rather than directly modifying unrelated component state.
- Testability: Components are structured so that their behavior can be tested independently where practical.

Architecture Constraints

- The system adopts a modular layered architecture as the approved architectural pattern for this project.
- The system is implemented in C/C++, using SFML for graphical rendering, with a console-based interface as an alternative build target.
- The system is a standalone desktop application with no network connectivity, no database, and no external service dependencies.
- The only supported input device is the keyboard, with fixed controls: Arrow keys (movement), P (pause/resume), R (restart), and Esc (exit).
- Snake movement is strictly grid-based, and an immediate reverse of the Snake's current direction is not permitted.
- Game State is held in memory only for the duration of a session; no persistent storage is introduced.
- The system supports a single user role (Player) only; no Administrator, Database, or Server component is introduced.
- The nine components listed in this document form the standard component vocabulary used throughout the project.

3.2 Stakeholders & Concerns

The stakeholders below are drawn from the Intended Audience defined in the Snake Game SRS, restated here with their architectural concerns so that the design decisions in this document can be justified against real stakeholder needs rather than assumed ones.

Stakeholder	Primary Concern(s)
Player	Responsive, predictable controls; clear and immediate feedback of Score and Game State; stable gameplay with no unexpected crashes.
Development Team	Clear separation between components (Game Controller, Input Handler, Snake Manager, Food Manager, Collision Manager, Score Manager, Game Renderer, Game State Manager, Difficulty Manager) so that each can be built, modified, and debugged independently.

Stakeholder	Primary Concern(s)
Testers / QA	An architecture that maps cleanly onto the SRS's functional, non-functional, and security requirement IDs, so that each component is independently testable and traceable to a test case.
Project Team Members	A single, consistently used architecture pattern, component list, and terminology across all four contributors' sections, so the SAD can be combined without major rewriting.
Faculty / Evaluators	A design that is demonstrably consistent with the approved SRS - same requirement IDs, same components, same Game States - with no invented functionality or contradicted decisions.

No administrator, operator, database-administrator, or backend-service stakeholder exists for this project, consistent with the single-user-role decision and the “no external actor” constraint.

3.3 Component (UML) Diagram

Figure 3.1 shows the UML component diagram of the Snake Game. The system is a standalone, offline, single-player desktop application, so the components are the nine conceptual modules named in the SRS, arranged in layers, plus the external SFML/Console runtime. There is no server, database or network component.

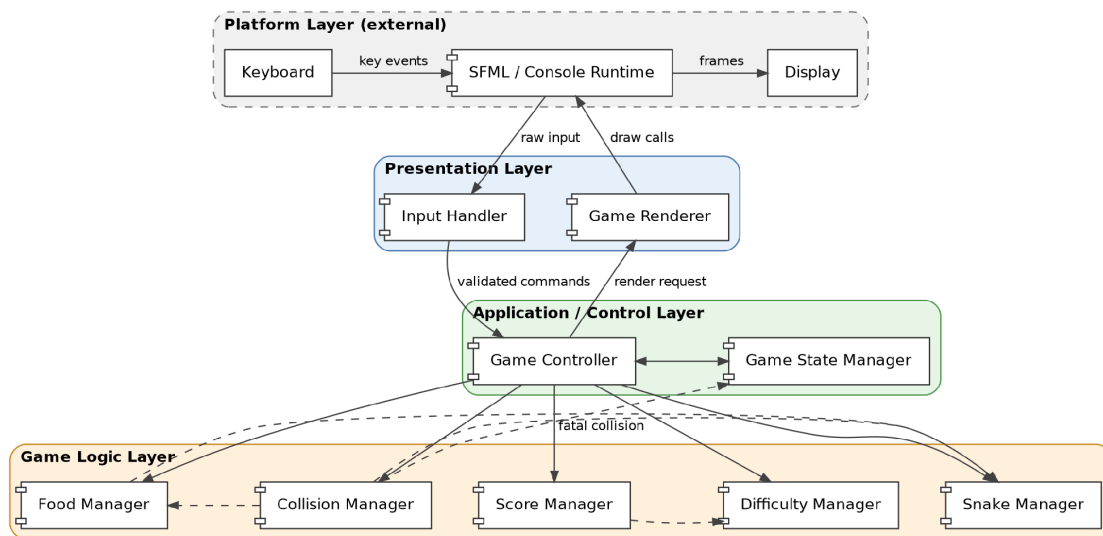


Figure 3.1 - Snake Game UML Component Diagram

How to read the diagram:

- Solid arrows are runtime calls or data flow; dashed arrows are read-only queries between logic components (e.g. Collision Manager reads the Snake and Food positions, Score Manager supplies the score to Difficulty Manager).
- Input flows from the Keyboard: Keyboard → Runtime → Input Handler → Game Controller. Output flows Game Controller → Game Renderer → Runtime → Display.
- Game Controller is the only component that drives the other managers, which keeps coupling low and supports SGNF-005 (maintainability).
- Game Renderer and Input Handler are the only components that touch the SFML/Console runtime, so the console fallback build changes only these two.

The diagram source is Graphviz DOT and can be redrawn in PlantUML or draw.io if the team prefers a different tool.

3.4 Component Descriptions

Each component below corresponds to a conceptual module in SRS Section 4.9 and to the Module column of the SRS Requirements Traceability Matrix.

Component	Responsibility	Interfaces / Depends on	Requirements
Input Handler	Captures raw keyboard events from the SFML/console runtime; validates	Runtime → Input Handler → Game Controller	SG-F-002, SG-F-004, SG-SR-001,

Component	Responsibility	Interfaces / Depends on	Requirements
	them and maps recognised keys (Arrow keys, P, R, Esc) to game commands. Rejects the immediate-reverse direction and ignores invalid or unrecognised keys. It never modifies Game State directly.		SG-SR-002, SG-SR-003
Game Controller	Central orchestrator and main game loop. Receives validated commands, calls the managers in the correct order each tick (move, collide, eat, score, speed), handles start, restart and exit, and initialises all game variables before play.	Uses all managers, Input Handler and Game Renderer	SG-F-001, SG-F-014, SG-F-015, SG-SR-003, SG-SR-005
Snake Manager	Owns the Snake Body and direction. Moves the Snake one grid cell per tick in its current direction and adds a segment when Food is eaten.	Queried by Collision Manager, Food Manager and Game Renderer	SG-F-003, SG-F-007
Food Manager	Generates Food at a random valid grid position inside the Game Board that does not overlap the Snake, and holds the current Food position.	Reads occupied cells from Snake Manager	SG-F-005
Collision Manager	Detects food collection (Snake Head on Food), Game Board boundary collision and self-collision (Snake Head on Snake Body).	Reads Snake Manager and Food Manager; reports collision result to Game Controller	SG-F-006, SG-F-009, SG-F-010
Score Manager	Keeps the current score, adds 10 points per Food collected, resets on restart, and supplies the score to the Renderer and the Difficulty Manager.	Feeds Difficulty Manager and Game Renderer	SG-F-008, SG-F-016
Difficulty Manager	Computes the Snake movement interval (speed) from the current score using the configured difficulty rule, so speed rises as score rises (with a cap for testability).	Reads Score Manager; used by Game Controller	SG-F-012
Game State Manager	Holds the current Game State (MENU, PLAYING, PAUSED, GAME OVER) and enforces legal transitions, including pause/resume with P and the move to GAME OVER on a fatal collision, as requested by Game Controller.	Controlled by Game Controller; read by Game Renderer	SG-F-011, SG-F-013, SG-F-017
Game Renderer	Draws the Main Menu, Gameplay Screen (board, Snake, Food, score, state and pause indicator) and Game Over Screen using SFML or the console. Read-only with respect to game data.	Reads Snake, Food, Score and State; draws through the Runtime	SG-F-016, SG-F-017, SGNF-001, SGNF-003
SFML / Console Runtime (external)	Third-party/platform library providing the graphical window, keyboard event handling, and drawing functionality.	Used by Input Handler and Game Renderer	SGNF-004

Edited from the original draft: the Collision Manager now reports its result to the Game Controller (rather than notifying the Game State Manager directly), and the Game State Manager's transitions are explicitly shown as requested by the Game Controller - this resolves a contradiction with Section 3.9, where state changes are described as routed solely through the Game Controller. The Runtime's description also no longer lists "timing," since only Input Handler and Game Renderer are said to touch the runtime, while the Game Controller owns tick timing via standard C++ facilities.

3.5 Chosen Architecture Pattern and Rationale

The Snake Game uses a modular layered architecture, built directly from the nine components already agreed for the project: Game Controller, Input Handler, Snake Manager, Food Manager, Collision Manager, Score Manager, Game Renderer, Game State Manager, and Difficulty Manager.

Layer breakdown

- Input Layer - Input Handler: captures keyboard input and validates it before it reaches game logic.
- Control / State Layer - Game Controller and Game State Manager: own the game's lifecycle (start, restart, exit) and the current Game State (MENU, PLAYING, PAUSED, GAME OVER).
- Core Game Logic Layer - Snake Manager, Food Manager, Collision Manager, Score Manager, Difficulty Manager: implement the actual gameplay rules (movement, growth, food placement, collision detection, scoring, speed scaling).
- Presentation Layer - Game Renderer: draws the Game Board, Snake, Food, Score, and Game State to the SFML window or console output.

Rationale

- Directly reuses the agreed component list without renaming or inventing components, so this section stays consistent with every other contributor's section.
- Each layer, and each component within a layer, can be built, modified, and unit-tested independently - directly satisfying the maintainability requirement (SGNF-005).
- Matches the small, standalone scope of the project: a single-player, offline desktop game does not need network, service, or persistence layers.
- Keeps a clear primary flow of control from Input through the Control/State layer to Core Logic and Presentation, while allowing defined read-only queries between components, making the design easy to reason about and to trace back to individual SRS requirement IDs.

Alternatives considered

- Monolithic single-loop design - rejected. Bundling input handling, game logic, and rendering into one block would make it difficult to modify one concern without risking the others, working against SGNF-005 and against the project's agreed component boundaries.
- Client-server / microservices architecture - rejected. This would introduce network and service infrastructure the project does not need or want, directly contradicting the SRS's "Network: Not required" and "Database: Not required" decisions.
- Entity-Component-System (ECS) - considered and rejected. ECS is suited to games with many dynamic entity types; the Snake Game has a small, fixed set of concerns that the team's existing module list already models cleanly, so adopting ECS would mean discarding the agreed component names for no real benefit.

A modular layered architecture was therefore chosen as the best fit: it keeps the project simple, reuses the agreed terminology and components exactly, and gives every contributor's module a clear, independent place in the overall design.

3.6 Technology Stack & Data Stores

The technology stack follows the project's fixed decisions exactly, as defined in the SRS.

Language: C/C++, used to implement all nine game components.

Graphics / Windowing: SFML (Simple and Fast Multimedia Library), used for rendering, window management, and keyboard input events in the graphical build.

Alternative interface: a console-based build target, used as the fallback presentation layer where an SFML/graphical environment is not available.

Compiler / IDE: a standard C/C++ compiler (e.g. g++ or an equivalent toolchain) together with any IDE or editor capable of building and linking against SFML (e.g. Code::Blocks, Visual Studio, or VS Code). The project does not mandate a specific IDE.

Standard libraries: the C++ Standard Library (STL) - for example, standard containers to represent the Snake's body segments, and the standard library's random-number facilities to generate valid Food positions.

Data stores

- None. The project uses no database and no external or persistent storage.
- All Game State - Snake position, Score, Food position, and the current Game State - is held entirely in memory for the duration of a single session and is not saved between runs.

External services

- None. No network libraries, no third-party API clients, and no cloud or backend SDKs are used.

3.7 Risks & Mitigations

Likelihood / Impact is rated Low, Medium or High. Risks follow the Test Plan risk list and add architecture-level risks.

ID	Risk	Likelihood / Impact	Mitigation
R1	SFML version or setup differs across team machines, causing build failures	Medium / High	Pin one SFML version in the README build steps; keep the console build as a fallback target (Runtime abstraction in Input Handler and Game Renderer).
R2	Input latency exceeds 100 ms (SGNF-001) because input, update and drawing run in one loop	Medium / High	Poll input every frame, decouple the render rate from the game-tick rate, keep per-tick work O(snake length), and verify with TC-PERF-01.
R3	Speed scaling (SG-F-012) makes the game unplayable or untestable at high scores	Medium / Medium	Define a minimum tick interval (maximum speed) in Difficulty Manager; allow a test override so input response can be tested independently.
R4	Grid off-by-one or boundary errors cause missed or false collisions	Medium / High	Use a single grid coordinate system owned by the Snake Manager; add boundary-value cases to TC-COLL-01 and TC-MOVE-02.
R5	Food generation fails to find a valid position, loops excessively, or places Food on the Snake when the board is nearly full	Low / High	Validate each candidate Food position against the Game Board and Snake before accepting it; if the board is nearly full, bound the number of attempts and end the game if no valid position can be found (TC-FOOD-01).
R6	Tight coupling between modules makes changes break other modules and violates SGNF-005	Medium / Medium	Layered structure with one-way dependencies; managers talk only through the Game Controller or narrow read-only interfaces; review in TC-MAINT-01.
R7	Invalid, repeated or unexpected key input corrupts Game State or crashes the game	Medium / High	Whitelist valid keys, one direction change per tick, ignore everything else (SG-SR-001 to 003); test with TC-SEC-01 to 03.
R8	State not fully reset on restart, or pause/resume leaves stale state	Medium / Medium	Single reset routine in Game Controller that re-initialises snake, score, speed, food and state (SG-F-014, SG-SR-005); test with TC-RESTART-01.
R9	Parallel work by four members causes integration bugs at merge time	Medium / Medium	Agree module interfaces in this document first; run integration tests after every merge into main.
R10	Game runs only on one OS or fails on the supported desktop environment (SGNF-004)	Low / Medium	Use only portable C/C++ and SFML APIs; avoid OS-specific calls; run TC-PORT-01 on the declared platform.

Edited from the original draft: R5 no longer commits to a specific food-placement algorithm (“list of free cells”), keeping it consistent with the implementation-neutral wording used in Sections 3.4 and 4.4.

3.8 Traceability to Requirements

The table traces every requirement in the SRS (17 functional, 5 non-functional and 5 security, 27 in total) to the SRS section, the architecture component that realises it, and its test case. It matches the SRS Section 8 RTM; Status is N (Not tested) for all rows until testing begins.

Req ID	Requirement	SRS Sec.	Component (Architecture)	Test Case
SG-F-001	Start new game	4.1	Game Controller	TC-START-01

Req ID	Requirement	SRS Sec.	Component (Architecture)	Test Case
SG-F-002	Accept directional keyboard input	4.2	Input Handler	TC-MOVE-01
SG-F-003	Move snake continuously	4.2	Snake Manager	TC-MOVE-02
SG-F-004	Prevent immediate reverse direction	4.2	Input Handler	TC-MOVE-03
SG-F-005	Generate food at valid random position	4.3	Food Manager	TC-FOOD-01
SG-F-006	Detect food collection	4.3	Collision Manager	TC-FOOD-02
SG-F-007	Increase snake length after food	4.3	Snake Manager	TC-GROW-01
SG-F-008	Increase score after food	4.3	Score Manager	TC-SCORE-01
SG-F-009	Detect Game Board boundary collision	4.4	Collision Manager	TC-COLL-01
SG-F-010	Detect self-collision	4.4	Collision Manager	TC-COLL-02
SG-F-011	Transition to Game Over on fatal collision	4.4	Game State Manager	TC-GAMEOVER-01
SG-F-012	Increase speed as score increases	4.5	Difficulty Manager	TC-SPEED-01
SG-F-013	Pause and resume gameplay	4.6	Game State Manager	TC-PAUSE-01
SG-F-014	Restart game after Game Over	4.7	Game Controller	TC-RESTART-01
SG-F-015	Exit the game	4.7	Game Controller	TC-EXIT-01
SG-F-016	Display current score	4.8	Score Manager / Game Renderer	TC-SCORE-02
SG-F-017	Display current Game State	4.8	Game State Manager / Game Renderer	TC-STATE-01
SGNF-001	Input response ≤ 100 ms	5	Input Handler / Game Renderer	TC-PERF-01
SGNF-002	Consistent Game State, no crashes	5	Entire System	TC-REL-01
SGNF-003	Clear, understandable controls/status	5	Game Renderer	TC-UX-01
SGNF-004	Runs on supported desktop environment	5	Entire System (SFML/Console Runtime)	TC-PORT-01
SGNF-005	Modular, maintainable code	5	All Modules / Codebase	TC-MAINT-01
SG-SR-001	Validate Player input before movement	5.1.2	Input Handler	TC-SEC-01
SG-SR-002	Ignore invalid/unrecognised input	5.1.2	Input Handler	TC-SEC-02
SG-SR-003	Prevent unsupported input from altering state	5.1.2	Game Controller / Input Handler	TC-SEC-03
SG-SR-004	No network transmission of gameplay data	5.1.2	Entire System	TC-SEC-04
SG-SR-005	Initialise game variables before gameplay	5.1.2	Game Controller	TC-SEC-05

Coverage check: all 27 requirements map to at least one component in Section 3.4, and every component in Section 3.4 realises at least one requirement.

3.9 Security Architecture

Security is scoped to a standalone game, as in SRS Section 5.1, rather than banking-grade controls. The security objectives are SG-SO-001 (protect the integrity of the game's internal state) and SG-SO-002 (prevent unsupported inputs or external interfaces from causing invalid behaviour).

Trust boundary and attack surface

- The only entry point is the keyboard, through the SFML/console runtime. Everything crossing the boundary from the runtime into Input Handler is treated as untrusted.
- There are no other entry points: no network, no files or database, no third-party APIs, no admin interface.
- All game data lives in memory for the session only.

Threat model (STRIDE)

STRIDE Category	Threat in the Snake Game	Mitigation in the architecture	Requirement
Spoofing	Fake or synthetic key events, or unrecognised commands, pretending to be valid Player input. There is no user identity or login, so identity spoofing does not apply.	Input Handler accepts only events delivered by the SFML/console runtime and only a fixed whitelist of keys (Arrow keys, P, R, Esc); everything else is ignored.	SG-SR-001, SG-SR-002
Tampering	Invalid input or a bad call changes Game State directly (e.g. an illegal reverse move, a forced state change, uninitialised variables).	State is changed only by Game Controller through the Game State Manager; the Input Handler validates and rejects reverse moves; all variables get defined initial values; the Renderer is read-only.	SG-SR-001, SG-SR-003, SG-SR-005, SG-F-004
Repudiation	Not applicable: no accounts, no multiplayer and no leaderboard, so there are no actions to deny. No audit log is required.	Accepted (out of scope per the SRS fixed decisions).	N/A
Information disclosure	Leak of gameplay or Player data. The game holds only score and board state in memory, with no personal data, no database and no network.	No network code and no file or database storage; nothing is transmitted or persisted.	SG-SR-004
Denial of service	Key flooding or rapid key mashing, bad input or a logic fault freezes or crashes the game; the board filling up causes an endless food search.	Bounded input handling (one direction change per tick, unknown keys dropped); fixed-size grid so snake length is bounded; Food positions are validated against the Game Board and Snake, with bounded attempts for generating a valid position; speed capped.	SGNF-002, SG-SR-002
Elevation of privilege	Not applicable in the usual sense: there is a single Player role and no privileged operation. The only risk is the game doing more than a normal user process should.	Runs as an ordinary user process; no admin interface, no shell or system calls, no external file access; Esc only closes the application.	Single role (SRS 2.3)

Edited from the original draft: the Denial of Service mitigation no longer says “food chosen from free cells only,” which implied a specific food-placement algorithm. It now matches the implementation-neutral wording used in Sections 3.7 (R5) and 4.4.

Security design principles applied

- Validate at the boundary: all input is validated in Input Handler before it reaches Game Controller.
- Single writer for state: only Game Controller, through Game State Manager, can change Game State.
- Fail safe: unknown input is dropped silently and does not change state.
- Least functionality: no network, storage or privileged operations exist to attack.

Residual risk: a local user with a debugger could alter process memory. This is accepted because the game has no assets to protect and no online features, consistent with the SRS scope.

4. Design

4.1 Design Overview

The Snake Game design translates the agreed modular layered architecture into the runtime design of a standalone, single-player desktop game. It uses the nine agreed components: Game Controller, Input Handler, Snake Manager, Food Manager, Collision Manager, Score Manager, Game Renderer, Game State Manager, and Difficulty Manager.

Layer	Components	Design responsibility
Input Layer	Input Handler	Captures keyboard events, validates supported input, rejects invalid/unrecognized commands and sends recognized commands to Game Controller.
Control / State Layer	Game Controller; Game State Manager	Coordinates the game loop, lifecycle operations and legal Game State transitions.
Core Game Logic Layer	Snake Manager; Food Manager; Collision Manager; Score Manager; Difficulty Manager	Implements grid-based movement, food placement, collision detection, snake growth, scoring and speed scaling.
Presentation Layer	Game Renderer	Displays the Main Menu, Game Board, Snake, Food, Score and Game State through SFML or the console-based interface.

Runtime Design and Per-Tick Game Loop

During active PLAYING gameplay, the Game Controller coordinates the repeated game tick. The logical order is: move → collide → eat → score → speed → render. Snake Manager moves the Snake by one grid cell according to the current direction. Collision Manager checks the Game Board boundary, self-collision and whether the Snake Head has reached Food. When Food is collected, Game Controller coordinates snake growth, score increase, speed update and generation of a new valid Food position. Game Renderer then displays the updated game data.

The Game Board is grid-based. Snake movement occurs from cell to cell, Food is placed at a valid unoccupied grid position, and boundary collision is determined from the Game Board limits.

Initialization and Game State Design

Before entering PLAYING, Game Controller initializes the Snake, Score, Food and Game State to their defined initial values. This supports the new-game initialization requirement and the security requirement that game variables have defined values before gameplay begins.

MENU → PLAYING ↔ PAUSED → GAME OVER

From GAME OVER: R (Restart) → PLAYING | Esc (Exit) → End

Game State Manager owns the current Game State and legal transitions. Game Controller coordinates lifecycle commands and requests the required state transition. Game Renderer reads the current Game State for display.

Design Principles

- Keep the nine agreed components separate with clear responsibilities.
- Validate Player input before it reaches gameplay processing and reject invalid or unrecognized input.
- Keep Game State changes controlled through Game Controller and Game State Manager.
- Keep Game Renderer read-only with respect to game data.
- Keep the design standalone and offline with no database, network/backend, or external service.

4.2 UML Sequence Diagrams

The following sequence diagrams show representative runtime interactions using the agreed component structure. Activation bars are shown on lifelines for active calls, and return messages are shown where the called component returns a result.

Figure 4.1 — Food Collection Sequence

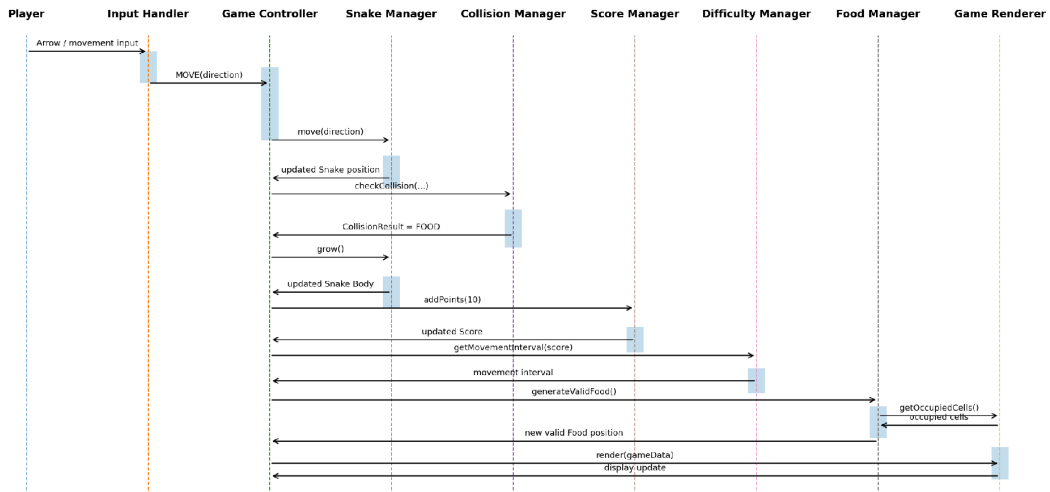


Figure 4.1 - Food Collection Sequence

The sequence includes movement, collision detection, snake growth (SG-F-007), score increase (SG-F-008), speed update (SG-F-012), valid Food replacement (SG-F-005), and rendering. Collision Manager returns a CollisionResult to Game Controller, which remains the coordinator.

Figure 4.2 — Game Over, Restart and Exit Sequence

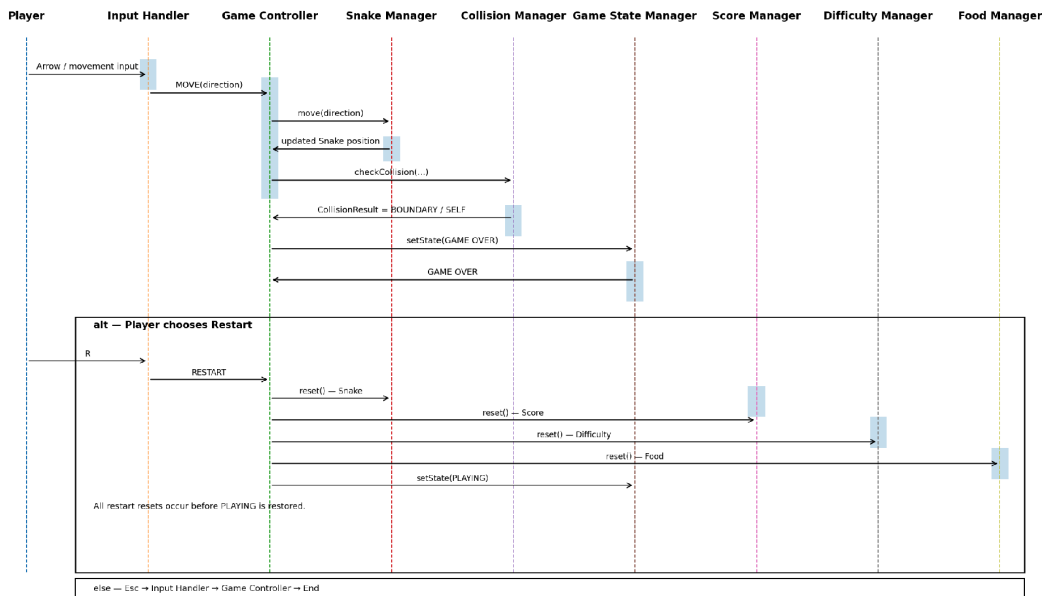


Figure 4.2 - Game Over, Restart and Exit Sequence

A fatal boundary/self-collision produces a collision result, after which Game Controller requests GAME OVER. For Restart, Snake, Score, Difficulty and Food are reset before PLAYING is restored. Esc is handled by Game Controller as the application-exit action.

4.3 API / Interface Design

Because the Snake Game is a standalone desktop application, this section defines internal component interfaces rather than REST, HTTP or external service APIs. The operations below describe logical calls between the agreed components.

Interface	Operation	Parameters	Return / Result	Error handling
Input Handler → Game Controller	getInputCommand()	Raw keyboard event	Command: MOVE / PAUSE / RESTART / EXIT / NONE	Invalid/unrecognized key → NONE; immediate reverse-direction input is ignored.
Game Controller → Snake Manager	move(direction)	Direction	Updated Snake position	Ignored unless Game State is PLAYING.
Game Controller → Collision Manager	checkCollision(snake, food, board)	Snake state; Food position; Game Board bounds	CollisionResult: NONE / FOOD / BOUNDARY / SELF	Result is returned to Game Controller; Collision Manager does not modify Game State.
Game Controller → Snake Manager	grow()	None	Updated Snake Body	Called only when CollisionResult = FOOD.
Game Controller → Score Manager	addPoints(points)	10	Updated Score	Score changes only through the approved food-collection flow.
Score Manager → Difficulty Manager	getMovementInterval(score)	Current Score	Movement interval / speed value	Uses the configured score-based speed-scaling rule.
Difficulty Manager → Game Controller	returnMovementInterval()	None	Current movement interval	Read-only result; no Game State modification.
Game Controller → Food Manager	generateValidFood()	None	New valid Food position	Food must be inside the Game Board and not overlap the Snake.
Food Manager → Snake Manager	getOccupiedCells()	None	Occupied Snake grid cells	Read-only query used for valid Food placement.
Game Controller → Game State Manager	setState(state)	MENU / PLAYING / PAUSED / GAME OVER	Updated Game State	Only legal transitions are accepted.
Game State Manager → Game Controller	getState()	None	Current Game State	Read-only access.
Game Controller → Game Renderer	render(gameData)	Snake; Food; Score; Game State	Display update	Renderer does not modify Game State or gameplay data.
Game Controller → Snake Manager	reset()	None	Initial Snake state	Used during Start/Restart initialization before PLAYING is restored.
Game Controller → Score Manager	reset()	None	Score = 0	Used during Start/Restart initialization before PLAYING is restored.
Game Controller → Difficulty Manager	reset()	None	Initial movement interval	Restores starting speed during Start/Restart.
Game Controller → Food Manager	reset()	None	Initial valid Food position	Used during Start/Restart to initialize Food before PLAYING; Food must not overlap the Snake.

Interface Constraints

- All interfaces are internal to the application; there is no network or backend communication.
- Input Handler validates supported keyboard input before commands reach Game Controller.
- Unsupported or unrecognized input cannot directly modify Game State.
- Game State is held in memory for the session and is controlled through Game Controller and Game State Manager.
- Game Renderer is read-only with respect to game data.
- The interfaces do not introduce functionality outside the approved SRS.

4.4 Error Handling, Logging & Monitoring

Error Handling

Condition	Handling Approach	Responsible Component
Invalid or unrecognized keyboard input	Input is validated before processing; unrecognized key presses are ignored and produce no change in Snake direction or Game State.	Input Handler
Immediate reverse-direction input	A direct opposite-direction input while the Snake is moving is detected and discarded rather than applied.	Input Handler
Invalid Food position	The generated Food position is validated to ensure it lies within the Game Board and does not overlap the Snake. If the position is invalid, the Food Manager selects another valid position.	Food Manager
Collision detected (boundary or self-collision)	The Collision Manager reports the fatal collision to the Game Controller, which requests the Game State Manager to transition the system to GAME OVER.	Collision Manager → Game Controller → Game State Manager
Invalid or unsupported Game State transition (e.g., movement input while in GAME OVER or MENU)	Transitions and input processing not valid for the current state are rejected, preventing unsupported input from directly modifying Game State.	Game State Manager
Uninitialized or undefined game variables at startup	All game variables (Snake position/length, Score, Food position, Game State) are initialized to defined values before gameplay begins.	Game Controller

Logging & Monitoring

As a standalone, offline, single-player desktop application, logging and monitoring are kept intentionally lightweight and local:

- Debug/console logging (e.g., state transitions, collision events, rejected input) may be written to the console or a local log output during development and testing.
- No gameplay or Player data is transmitted over a network for logging or monitoring purposes.
- Logging is a development and debugging aid only, and does not substitute for the error-handling behavior defined above.

4.5 UX Design

The UX is designed for the single Player using the fixed keyboard controls defined in the SRS. The graphical build uses SFML. The interface keeps Score and Game State visible and communicates the available action for each Game State. The Main Menu presents Start Game and Exit; the SRS does not specify a keyboard key for activating Start Game, so this document does not invent one.

Screen / State	Displayed information	Player interaction
Main Menu	Game title; Start Game; Exit.	Player selects Start Game or Exit. No keyboard key for Start Game is invented because the SRS does not specify one.
Gameplay Screen	Game Board, Snake, Food, current Score, current Game State.	Arrow Keys control movement; P pauses/resumes; Esc exits.
Gameplay Screen - Paused	Game Board/game context, current Score, and visible PAUSED Game State.	P resumes gameplay; Esc exits.
Game Over Screen	GAME OVER message; Final Score; Restart option; Exit option.	R restarts the game after Game Over; Esc exits.

Edited from the original draft: the “Paused” row is now labelled “Gameplay Screen - Paused” rather than a standalone “Paused State,” since the SRS defines three screens only (Main Menu, Gameplay Screen, Game Over Screen); Paused is a Game State shown as an overlay on the Gameplay Screen (see Figure 4.5), not a fourth distinct screen.

Keyboard Controls

Key	Action
Arrow Keys	Control the Snake's movement direction.
P	Pause or resume gameplay.
R	Restart the game after Game Over.

Key	Action
Esc	Exit the game.

Screen Mockups



Figure 4.3 - Main Menu

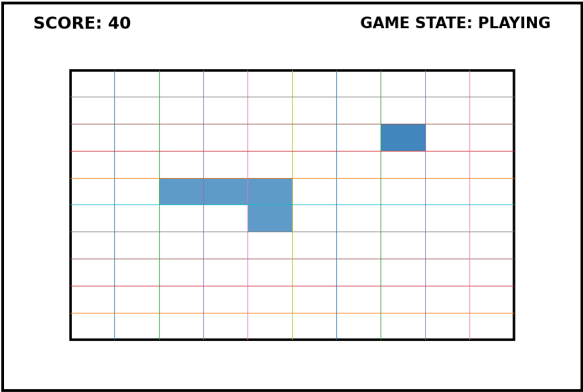


Figure 4.4 - Gameplay

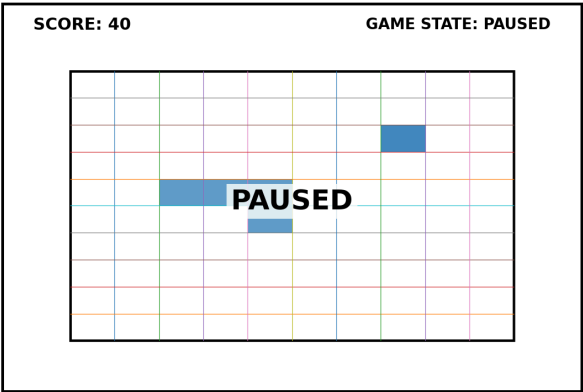


Figure 4.5 - Gameplay Screen, Paused



Figure 4.6 - Game Over

4.6 Open Issues & Next Steps

The items below are potential future enhancements only. None of them are part of the current scope, none modify any existing SG-F, SGNE, or SG-SR requirement, and none should be treated as current requirements. Adopting any of these would require updating the master decision sheet and informing the group first.

Open Issue / Future Idea	Notes
Persistent high-score storage	Would require reconsidering the project's "No database" decision - out of scope for v1.0.
Configurable difficulty presets	An optional Easy/Medium/Hard selection layered on top of the current fixed speed-scaling rule (SG-F-012) - not part of the current difficulty model.
Alternative input devices	Potential gamepad/controller support in addition to the keyboard, if supported by SFML - the current SRS fixes keyboard as the only input device.
Visual polish	Additional animations or visual themes for the SFML build - purely cosmetic, with no effect on any functional or non-functional requirement.

These are recorded only as possible directions for a future iteration of the project, beyond the scope of the current, approved SRS v1.0.